

CÀI ĐẶT PERCEPTRON VÀ MẠNG NƠ-RON NHIỀU LỚP (MLP) TỪ ĐẦU BẰNG NUMPY

Đại học Phenikaa
Trường Công nghệ Thông tin Phenikaa

Giảng viên hướng dẫn: ThS. Phan Thị Hoài

Sinh viên thực hiện:
Ngô Quang Thiện - Nguyễn Hưng Vũ

Nội dung trình bày

- 1 Cơ sở lý thuyết
- 2 Kết quả thực nghiệm
- 3 Thực nghiệm mở rộng

2.1 Quy ước ký hiệu

- X : Ma trận dữ liệu đầu vào. x : Vector đặc trưng.
- $W^{(l)}$: Ma trận trọng số lớp l . w : Vector trọng số.
- $b^{(l)}$: Vector bias lớp l .
- z : Giá trị trước hàm kích hoạt. $Z^{(l)}$: Ma trận giá trị trước kích hoạt.
- a : Đầu ra sau hàm kích hoạt. $A^{(l)}$: Ma trận đầu ra sau kích hoạt.
- $\hat{y}$: Giá trị dự đoán. y : Nhãn thực tế.
- $\delta^{(l)}$: Sai số lan truyền ngược lớp l .
- η : Tốc độ học. β : Hệ số Momentum. λ : Hệ số L2.
- $\mathcal{L}$: Hàm mất mát.

2.2.1 Kiến trúc Perceptron

- **Đầu vào:** $x = [x_1, x_2, \dots, x_n]^T$
- **Trọng số:** $w = [w_1, w_2, \dots, w_n]^T$
- **Tổng có trọng số:**

$$z = \sum_{i=1}^n w_i x_i + b \quad \text{hay} \quad z = w^T x + b$$

- **Hàm kích hoạt (Step Function):**

$$\hat{y} = f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

- **Cập nhật trọng số:**

$$w_i = w_i + \eta(y - \hat{y})x_i, \quad b = b + \eta(y - \hat{y})$$

2.2.2 Giới hạn phân tách tuyến tính (Bài toán XOR)

- Perceptron chỉ phân tách được các dữ liệu tuyến tính (có thể chia bằng 1 đường thẳng/mặt phẳng).
- **Bài toán XOR (Exclusive OR):**

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

- Dữ liệu XOR phân bố chéo nhau, không thể dùng một đường thẳng phân tách. Do đó Perceptron một lớp không thể giải bài toán này.

2.2.3 Mạng nơ-ron nhiều lớp (MLP) & Cách giải quyết XOR

- **Mạng nơ-ron nhiều lớp (MLP):** Khắc phục hạn chế của Perceptron bằng cách thêm một hoặc nhiều **lớp ẩn (Hidden Layers)** giữa lớp Input và Output.
- Kết hợp với các **hàm kích hoạt phi tuyến** (như ReLU, Sigmoid), MLP có thể tạo ra các biến đổi không gian phức tạp.
- **Cách MLP giải quyết XOR:**
 - Các nơ-ron trong lớp ẩn học cách tạo ra nhiều ranh giới phân tách cục bộ.
 - Khi kết hợp lại ở lớp Output, chúng tạo thành một **ranh giới quyết định phi tuyến**, cho phép phân tách hoàn toàn các mẫu XOR mà Perceptron đơn lớp thất bại.

2.3.1 Hàm Sigmoid

- Biến đổi giá trị đầu vào về khoảng $(0, 1)$. Thường dùng cho bài toán phân loại nhị phân.
- **Công thức hàm:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Đạo hàm:**

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

2.3.2 Hàm ReLU

- **ReLU (Rectified Linear Unit):** Hàm kích hoạt phổ biến ở các Hidden Layer, giúp giảm thiểu hiện tượng mất gradient và tính toán nhanh.
- **Công thức hàm:**

$$f(z) = \max(0, z)$$

- **Đạo hàm:**

$$f'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

2.3.3 Hàm Softmax

- Dùng ở Output Layer của bài toán phân loại nhiều lớp. Chuyển vector đầu vào thành phân phối xác suất có tổng bằng 1.
- **Công thức tại nơ-ron c :**

$$\hat{y}_c = A_c^{(L)} = \frac{e^{z_c^{(L)}}}{\sum_{j=1}^C e^{z_j^{(L)}}}$$

- **Dạng ma trận:**

$$A^{(L)} = \text{Softmax}(Z^{(L)})$$

2.4 Lan truyền thuận (Forward Propagation)

- Truyền dữ liệu từ Input Layer $\rightarrow$ Hidden Layer $\rightarrow$ Output Layer.
- Ở mỗi lớp l , tính tổng có trọng số và áp dụng hàm kích hoạt.
- **Công thức tổng quát cho lớp l :**

$$Z^{(l)} = W^{(l)}A^{(l-1)} + b^{(l)}$$

$$A^{(l)} = f\left(Z^{(l)}\right)$$

2.5 Hàm mất mát (Loss Function)

- Đo lường sai lệch giữa dự đoán và thực tế.
- **Mean Squared Error (Hồi quy):**

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Binary Cross-Entropy (Phân loại nhị phân):**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Categorical Cross-Entropy (Phân loại nhiều lớp):**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$$

2.6 Lan truyền ngược (Backpropagation)

- Quy tắc đạo hàm dây chuyền: $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$
- Đạo hàm lớp cuối: $\delta^{(L)} = A^{(L)} - Y$
- Lan truyền sai số về lớp l :

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(Z^{(l)})$$

- Tính Gradient (trên N mẫu):

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{N} \delta^{(l)} (A^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{N} \sum_{i=1}^N \delta_i^{(l)}$$

2.7 Các thuật toán tối ưu (Optimization Algorithms)

- **Stochastic Gradient Descent (SGD):**

$$W^{(l)} = W^{(l)} - \eta \frac{\partial \mathcal{L}_i}{\partial W^{(l)}}, \quad b^{(l)} = b^{(l)} - \eta \frac{\partial \mathcal{L}_i}{\partial b^{(l)}}$$

- **SGD with Momentum:** Tích lũy gradient các bước trước.

$$v_t^{W^{(l)}} = \beta v_{t-1}^{W^{(l)}} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$
$$W_t^{(l)} = W_{t-1}^{(l)} + v_t^{W^{(l)}}$$

2.8 Regularization L2

- Thêm phần phạt vào hàm mất mát để chống Overfitting (Weight Decay).
- **Hàm mất mát mới:**

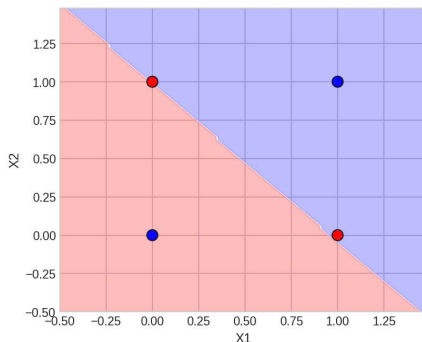
$$\mathcal{L}' = \mathcal{L} + \frac{\lambda}{2N} \sum_{l=1}^L \|W^{(l)}\|_F^2$$

- **Cập nhật trọng số:**

$$W^{(l)} = W^{(l)} - \eta \left(\frac{\partial \mathcal{L}}{\partial W^{(l)}} + \frac{\lambda}{N} W^{(l)} \right)$$

3.1 Cài đặt Perceptron đơn giản và bài toán XOR

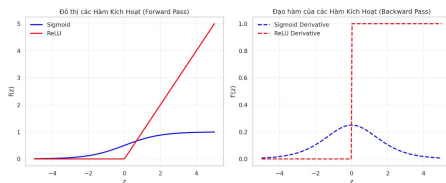
- Huấn luyện Perceptron 1 lớp với Step Function.
- Mô hình **không hội tụ**, Loss không giảm về 0, độ chính xác 75%.
- Perceptron chỉ vẽ được 1 đường thẳng tuyến tính, không tách được (1, 1).



Hình: Perceptron trên XOR

3.2 Đánh giá kết quả cài đặt thuật toán MLP

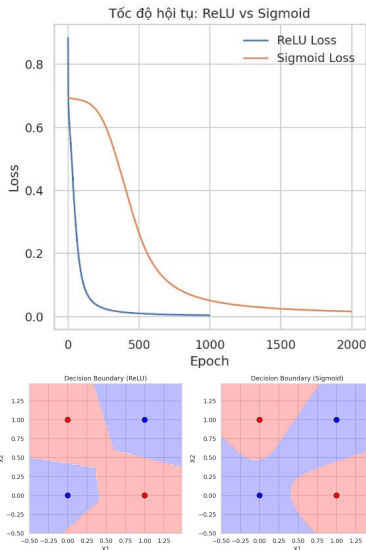
- Cài đặt MLP bằng NumPy. Hỗ trợ tùy chỉnh số lớp, số nơ-ron.
- Hỗ trợ Forward Propagation, Backpropagation bằng ma trận.
- Các hàm kích hoạt: Sigmoid, ReLU, Softmax hoạt động đúng như thiết kế toán học.



Hình: Các hàm kích hoạt & đạo hàm

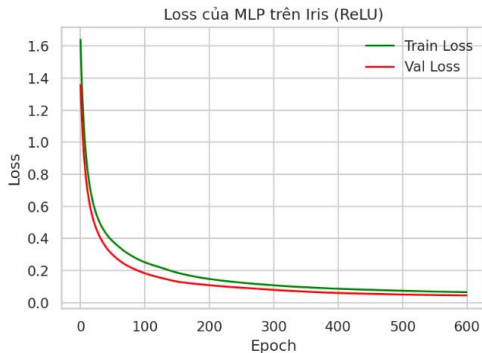
3.3 Đánh giá trên bài toán XOR

- **Kiến trúc:** 2 Input - 4 Hidden - 1 Output.
- **Kết quả:** Phân loại đúng 100%. MLP khắc phục được giới hạn tuyến tính, tạo ra ranh giới phi tuyến.
- Hàm ReLU hội tụ nhanh hơn Sigmoid.



3.4 Đánh giá trên tập dữ liệu Iris

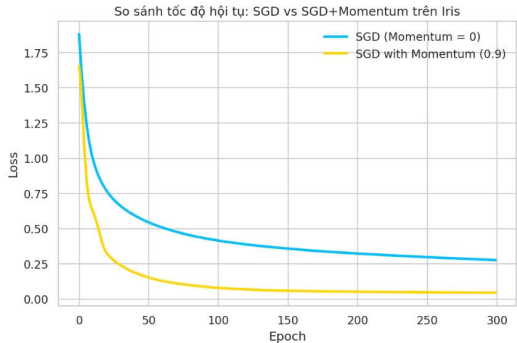
- **Kiến trúc:** 4 Input - 8 Hidden (ReLU) - 3 Output (Softmax).
- Dữ liệu chuẩn hóa, One-hot nhãn, chia Train(80%)/Test(20%).
- **Kết quả:** Hội tụ sau 600 epochs. Độ chính xác đạt 96% - 100%.



Hình: Huấn luyện MLP trên Iris

4.1 Đánh giá hiệu quả của thuật toán tối ưu SGD với Momentum

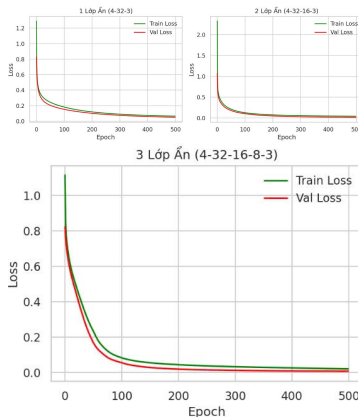
- So sánh SGD và SGD with Momentum ($\beta = 0.9$).
- **Kết quả:** Momentum giảm Loss nhanh hơn rất nhiều, đạt khoảng 0.05 sau 100 epochs, trong khi SGD mất 300 epochs chỉ để về 0.18.



Hình: SGD vs SGD+Momentum

4.2 Ảnh hưởng của số lượng lớp ẩn (Hidden Layers)

- **1 lớp:** Hội tụ chậm.
- **2 lớp (4-32-16-3):** Nhanh & tối ưu nhất.
- **3 lớp:** Cải thiện cực ít nhưng tốn tài nguyên.



4.3 Đánh giá hiệu quả của L2 Regularization

- Dùng mạng lớn (4-64-32-3) trên dữ liệu ít (30 mẫu) để ép Overfitting.
- **Không L2:** Train Loss $\rightarrow 0$, Val Loss cao (Overfitting).
- **Có L2 ($\lambda = 0.15$):** Val Loss giảm tiếp, thu hẹp khoảng cách. L2 chống Overfitting tốt.

